

Debounce in JavaScript — Detailed Explanation

Definition

Debounce is a technique that ensures a function is executed only after a specified period of inactivity. When repeated events occur (typing, scrolling, resizing), debouncing postpones the function call until the events stop for the specified delay. This prevents unnecessary or expensive operations from running too frequently.

Short summary

- Debounce delays function execution until events stop for *delay* milliseconds.
- Repeated calls within the delay window cancel the previously scheduled call and reschedule a new one.
- Only the last call (after the pause) actually runs the function.

Common Use Cases

- Search box API requests (call API once user stops typing).
- Window resize handlers (run layout logic after resize stops).
- Autocomplete/typeahead input throttling to reduce network requests.
- Autosave forms (save after the user pauses typing).
- Preventing button spamming (ignore rapid repeated clicks).

Debounce: Basic Implementation

```
function debounce(func, delay) {  
  let timer;  
  return function (...args) {  
    clearTimeout(timer);  
    timer = setTimeout(() => {  
      func.apply(this, args);  
    }, delay);  
  };  
}
```

Detailed line-by-line explanation

1. `let timer;`

- Declares a variable in the surrounding scope (closure). It persists across calls to the returned function.

2. `return function (...args) { ... }`

- Returns a wrapper function that will be called on each event (e.g., each keystroke).
- We use a normal function (not an arrow) so that the wrapper receives a dynamic `this` and can forward it to `func`.

3. `clearTimeout(timer);`

- Cancels the previously scheduled timeout (if any). If `timer` is undefined or already expired, this does nothing.
- This is what prevents the earlier scheduled calls from running when events continue to arrive.

4. `timer = setTimeout(() => { func.apply(this, args); }, delay);`

- Schedules a new call for `delay` milliseconds in the future and stores the timer id in `timer`.
- If another call happens before `delay` elapses, the scheduled call will be cleared and replaced with a new one.

5. `func.apply(this, args);`

- When the timer callback runs, it invokes the original function `func` with the most recent arguments and the correct `this` context.

Concrete timeline example (with timer ids)

Assume `delay = 500ms`. Calls happen at `t=0ms`, `t=300ms`, `t=700ms`.

- t=0ms: first call
 - timer is undefined initially.
 - setTimeout schedules execution at $0 + 500 = 500\text{ms}$. Suppose setTimeout returns id = 101.
 - timer = 101.
- t=300ms: second call (within 500ms)
 - clearTimeout(101) cancels the scheduled callback that was to run at 500ms.
 - setTimeout schedules execution at $300 + 500 = 800\text{ms}$. Suppose id = 102.
 - timer = 102.
- t=700ms: third call (within 800ms)
 - clearTimeout(102) cancels scheduled callback at 800ms.
 - setTimeout schedules execution at $700 + 500 = 1200\text{ms}$. Suppose id = 103.
 - timer = 103.
- No more calls → the last scheduled callback executes at t = 1200ms, calling func once.

Note about resetting `timer` after execution

After the timeout callback runs, `timer` still holds the id value in many implementations. This is harmless because:

- The timer has already executed (it's not pending any more). `clearTimeout(timer)` will do nothing the next time. If you want to track whether a timeout is pending, set `timer = null` (or `undefined`) inside the callback:

```
timer = setTimeout(() => {
  func.apply(this, args);
  timer = null; // now timer indicates that nothing is pending
}, delay);
```

Leading (immediate) + Trailing option

Sometimes you want the function to run immediately on the first call (leading) and then suppress until the burst ends (trailing). Here's a version supporting both options:

```
function debounce(func, delay, { leading = false } = {}) {
  let timer;
  return function (...args) {
    const callNow = leading && !timer;
    clearTimeout(timer);
    timer = setTimeout(() => {
      timer = null;
      if (!leading) func.apply(this, args); // trailing call when not leading
    }, delay);
    if (callNow) func.apply(this, args); // leading immediate call
  };
}
```

Practical example: Debounced Search (API)

Wrap your API-calling function with debounce, so it runs only after the user stops typing:

```
const debouncedSearch = debounce(async (query) => {
  if (!query) return;
  const res = await fetch('https://api.example.com/search?q=' + encodeURIComponent(query));
  // handle results
}, 400);

inputEl.addEventListener('input', (e) => {
  debouncedSearch(e.target.value);
});
```

Key points:

- Use the returned function (debouncedSearch) as the event handler – it preserves the reference so you can call it again.
- Choose delay based on UX vs network cost (300–700ms common for search).

Gotchas & Tips

- **this context**: Use a normal function for the returned wrapper (not arrow) if you need to preserve `this` for methods. - **Removing listeners**: Keep a reference to the debounced function to call `removeEventListener` later. - **Resetting timer**: If you need to check whether a call is pending, set `timer = null` inside the callback. - **Node vs Browser**: `setTimeout` returns different types (object in Node, number in browsers) but `clearTimeout` works with both. - **Debounce vs Throttle**: Debounce waits until inactivity; throttle ensures a function runs at most once per interval.

End of document — detailed debouncing overview